Звіт з лабораторної роботи №10

Рівненський фаховий коледж інформаційних технологій · Дисципліна: Об'єктно-орієнтоване програмування

Виконав: Зінчук Д. Ю., група ІПЗ 3/1 · **Варіант:** 7 (`INotifier` та `Authenticator`)

Мета роботи

Засвоїти використання інтерфейсів та абстрактних класів у C#, реалізувати поліморфну поведінку через колекції інтерфейсного й абстрактного типів.

Завдання варіанта 7

- Інтерфейс `INotifier` з методом `void SendNotification(string message)`.
- Реалізації інтерфейсу: `EmailNotifier`, `SmsNotifier`.
- Абстрактний клас `Authenticator`: абстрактний метод `bool Authenticate(string username, string password)` та конкретний метод `void LogAttempt(string username, bool success)`.
- Похідні класи: `DatabaseAuthenticator`, `LdapAuthenticator`.

Хід роботи

1. Створено консольний проєкт `lab10v7` командою `dotnet new console -o lab10v7`.
2. Описано інтерфейс `INotifier` і два класи, що його реалізують.
3. Описано абстрактний клас `Authenticator` з абстрактним методом `Authenticate` та конкретним `LogAttempt`.
4. Створено похідні класи `DatabaseAuthenticator` та `LdapAuthenticator`, які перевизначають `Authenticate` через `override`.
5. У методі `Main` створено `List<INotifier>` та `List<Authenticator>`, пройдено по них циклами й викликано методи.
6. Результати виведено в консоль.

Структура проєкту

```
lab10v7/
├── INotifier.cs
├── EmailNotifier.cs
├── SmsNotifier.cs
├── Authenticator.cs
├── DatabaseAuthenticator.cs
├── LdapAuthenticator.cs
└── Program.cs
```

Опис типів

Тип	Вид	Призначення
`INotifier`	Інтерфейс	Контракт: `SendNotification(string message)`
`EmailNotifier`	Клас	Виводить повідомлення у форматі `[Email -> адреса]`
`SmsNotifier`	Клас	Виводить повідомлення у форматі `[SMS -> номер]`
`Authenticator`	Абстрактний клас	Абстрактний `Authenticate`, спільний `LogAttempt`
`DatabaseAuthenticator`	Клас-нащадок	Перевірка за імітацією таблиці БД
`LdapAuthenticator`	Клас-нащадок	Перевірка за імітацією каталогу LDAP

Ключові фрагменти коду

```
public interface INotifier
{
    void SendNotification(string message);
}

public abstract class Authenticator
{
    public abstract bool Authenticate(string username, string password);

    public void LogAttempt(string username, bool success)
    {
        string result = success ? "успішно" : "невдало";
        Console.WriteLine($"Спроба входу '{username}': {result}");
    }
}
```

Результат роботи

Програма виводить два сповіщення (Email і SMS) через однаковий виклик `SendNotification`. Далі для кожного автентифікатора з колекції викликається `Authenticate`: `admin / admin123` проходить лише в `DatabaseAuthenticator`, `ivan / ldap2026` лише в `LdapAuthenticator`, а `admin / wrong` не проходить в обох. Кожна спроба фіксується спільним методом `LogAttempt`.

Рис. 1 — вивід програми `dotnet run --project lab10v7` (вставити скріншот консолі).

Висновки

У ході роботи створено консольний проєкт `lab10v7` з інтерфейсом `INotifier`, його реалізаціями `EmailNotifier` та `SmsNotifier`, абстрактним класом `Authenticator` і похідними `DatabaseAuthenticator` та `LdapAuthenticator`.

Інтерфейс задає контракт, який кожен клас виконує по-своєму, а абстрактний клас поєднує спільну логіку (`LogAttempt`) з методом `Authenticate`, що реалізують нащадки. Робота з колекціями `List<INotifier>` і `List<Authenticator>` показала поліморфізм: один виклик через базовий тип дає різну поведінку залежно від реального класу об'єкта. Це дозволяє додавати нові способи сповіщення чи автентифікації без змін у коді, який їх використовує.

Посилання на репозиторій

https://github.com/zinchukdiipz25-wq/OOP---Zinchuk-labs